

Game Engine and Game Development Report

William Hammond

Chapter 1: Game Engine Design and Critique

1.1 Introduction

This project is a first-person maze survival game developed in C# using OpenTK and OpenGL. The player moves through a 3D maze while avoiding hostile drones, collecting power-ups, and trying to survive long enough to defeat or disable the enemies.

The final intended design uses an Entity Component System style architecture. The purpose of this was to separate reusable engine behaviour from game-specific behaviour. In this structure, an entity is a general object container, components hold data, and systems process entities that contain the required components.

This design was chosen because the game contains objects that share some features but not all features. For example, walls need rendering and collision but do not need movement. Drones need rendering, collision and position data, but their pathfinding is handled by a drone service. Power-ups need rendering, position and collision, but they do not need AI. The ECS design was not fully completed for final submission because of poor time management during the refactor. However, the work demonstrates the intended engine structure and shows how the original project could be improved by separating rendering, movement, collision, input, audio, highscore communication and gameplay services.

1.2 ECS UML Diagrams

The report should include UML diagrams showing the class relationships in the final ECS design. These diagrams should not include member variables or methods. They should only show relationships such as inheritance, composition, associations and dependencies.

Figure 1: ECS Framework Overview

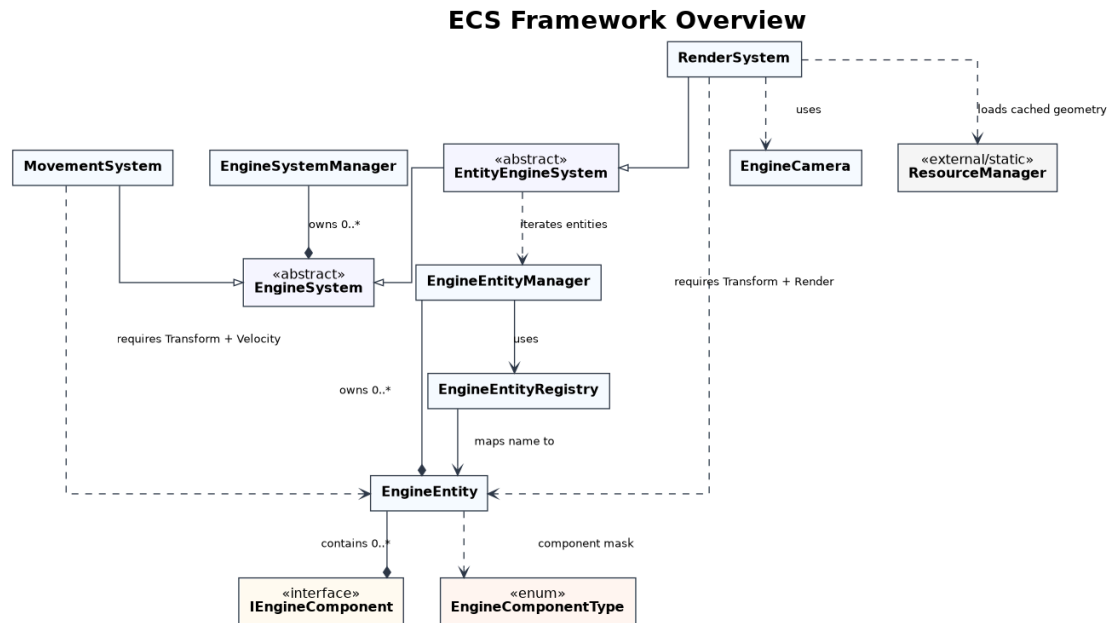
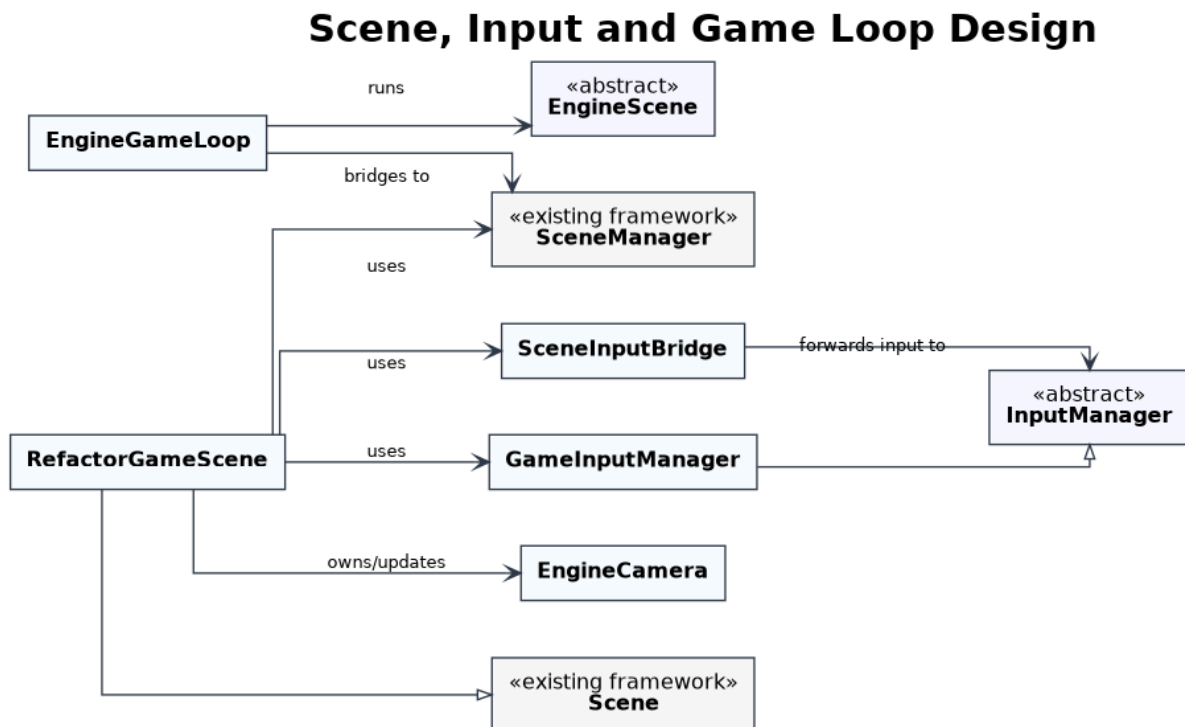


Figure 2: Scene, Input and Game Loop Design



1.3 Main Engine Class Descriptions

Class	Role	Justification
EngineEntity	Represents a game object in the ECS framework.	Keeps objects generic and avoids a large inheritance hierarchy.
IEngineComponent	Base interface for components.	Allows different component types to be stored together.
EngineComponentType	Enum for identifying component types.	Enables compact component masks for checking entity composition.
TransformComponent	Stores position, rotation and scale.	Almost all world objects need transform data.
VelocityComponent	Stores velocity.	Allows moving objects to be updated by the movement system.
AabbCollisionComponent	Stores AABB collision bounds.	Separates collision data from rendering and gameplay logic.
RenderComponent	Stores geometry, shader and texture references.	Allows renderable objects to be handled consistently.
ScaleComponent	Stores scale values.	Allows object size to be changed without needing object-specific classes.

TagComponent	Stores a general tag or name.	Helps identify entities without depending on class type.
GameplayTagComponent	Stores gameplay categories such as wall, drone or power-up.	Keeps game-specific classification separate from the entity class.
AudioComponent	Stores OpenAL audio source information.	Keeps audio data separate from scene logic.
EngineEntityManager	Stores and manages entities.	Centralises entity ownership.
EngineEntityRegistry	Provides name-based entity lookup.	Avoids repeatedly searching through all entities manually.
EngineSystem	Abstract base class for systems.	Gives all systems a shared update structure.
EntityEngineSystem	Base class for systems that process entities.	Reduces repeated entity-looping code.
MovementSystem	Applies velocity to transform using delta time.	Separates movement behaviour from object classes.
RenderSystem	Draws entities that have render and transform data.	Keeps rendering in one reusable system.
EngineCamera	Stores camera position, view and projection data.	Separates camera maths from scene logic.

InputManager	Stores key and mouse input states.	Creates a reusable input layer.
GameInputManager	Converts raw input into game actions.	Keeps gameplay input readable and easier to change.
EngineScene	Abstract base for engine-driven scenes.	Gives scenes a consistent update, render and close structure.
EngineGameLoop	Connects the engine scene to the existing scene manager.	Allows the refactor to work with the older framework.

1.4 Game-Specific Service Descriptions

Class	Role	Justification
RefactorGameScene	Main refactored gameplay scene.	Integrates the ECS framework with the actual game.
RefactorEntityFactory	Creates common ECS entities.	Reduces repeated setup code for walls, drones and power-ups.
DroneService	Handles drone pathfinding and movement.	Keeps AI logic separate from the scene.
PowerUpService	Handles power-up collection and effects.	Keeps power-up behaviour modular.

EngineWorldCollisionService	Resolves player movement against wall colliders.	Keeps collision response separate from input.
EngineEntityQueryService	Provides helper functions for finding entities and positions.	Reduces repeated component lookup code.
HudRenderService	Draws HUD information such as lives and score.	Keeps UI rendering separate from game logic.
AudioService	Registers and plays sounds by name.	Hides low-level OpenAL details.
HighscoreClient	Communicates with the highscore server.	Keeps networking separate from gameplay.
EngineMazeBuilder	Converts maze grid data into wall entities.	Separates maze construction from the scene.
GameSettings	Stores configurable gameplay values.	Allows values to be changed without hard-coding.
GameSessionState	Stores score and game result data.	Allows results to be passed between scenes.

1.5 Entity Composition Matrix

Entity	Transform	Velocity	AABB Collision	Render	Scale	Tag	Gameplay Tag	Audio
--------	-----------	----------	-------------------	--------	-------	-----	-----------------	-------

Player	Yes	Yes	Yes	No	No	No	No	No
Skybox	Yes	No	No	Yes	Yes	Yes	Yes	No
Floor	Yes	No	Yes	Yes	Yes	Yes	Yes	No
Maze Wall	Yes	No	Yes	Yes	Yes	Yes	Yes	No
Drone	Yes	No	Yes	Yes	Yes	Yes	Yes	No
Health Power-Up	Yes	No	Yes	Yes	Yes	Yes	Yes	No
Weapon Power-Up	Yes	No	Yes	Yes	Yes	Yes	Yes	No
EMP Power-Up	Yes	No	Yes	Yes	Yes	Yes	Yes	No
Audio Source	No	No	No	No	No	No	No	Yes

This matrix shows why ECS is useful for this game. Each entity only owns the components it needs. A wall does not need velocity, a player does not need a render component in first-person view, and an audio source does not need collision or rendering.

1.6 Memory Efficiency

The engine uses several memory-conscious design choices. Entities store only the components they need, rather than every entity containing fields for rendering, collision, velocity, audio and gameplay data. This avoids wasting memory on unused data. The `EngineComponentType` enum uses flags, allowing an entity to store its component composition as a compact bitmask. This is more efficient than storing many separate boolean values and also makes it faster to

check whether an entity contains certain component types. Many manager and service references are declared as readonly. This makes ownership clearer and prevents accidental reassignment after construction. The engine also uses OpenTK value types such as Vector3, Matrix4 and Quaternion for mathematical data, which avoids unnecessary heap allocation.

Rendering is also designed to reduce repeated loading. Geometry is loaded through the resource manager, allowing the same model file to be reused instead of loaded multiple times. The refactored render system also caches shaders so that new shader programs are not repeatedly created during the game loop. This is important because repeatedly creating OpenGL resources can waste memory and cause performance problems.

Audio is handled through AudioService and AudioComponent. Audio buffers are loaded through the resource manager, and the service releases audio sources when the scene closes. This reduces the risk of leaving OpenAL resources active after the scene has ended.

The main weakness is that component lookup is still simple. Components are stored in a list, so finding a component requires searching through that list. This is acceptable for a small game, but a larger ECS would benefit from indexed component storage, component pools or arrays grouped by component type.

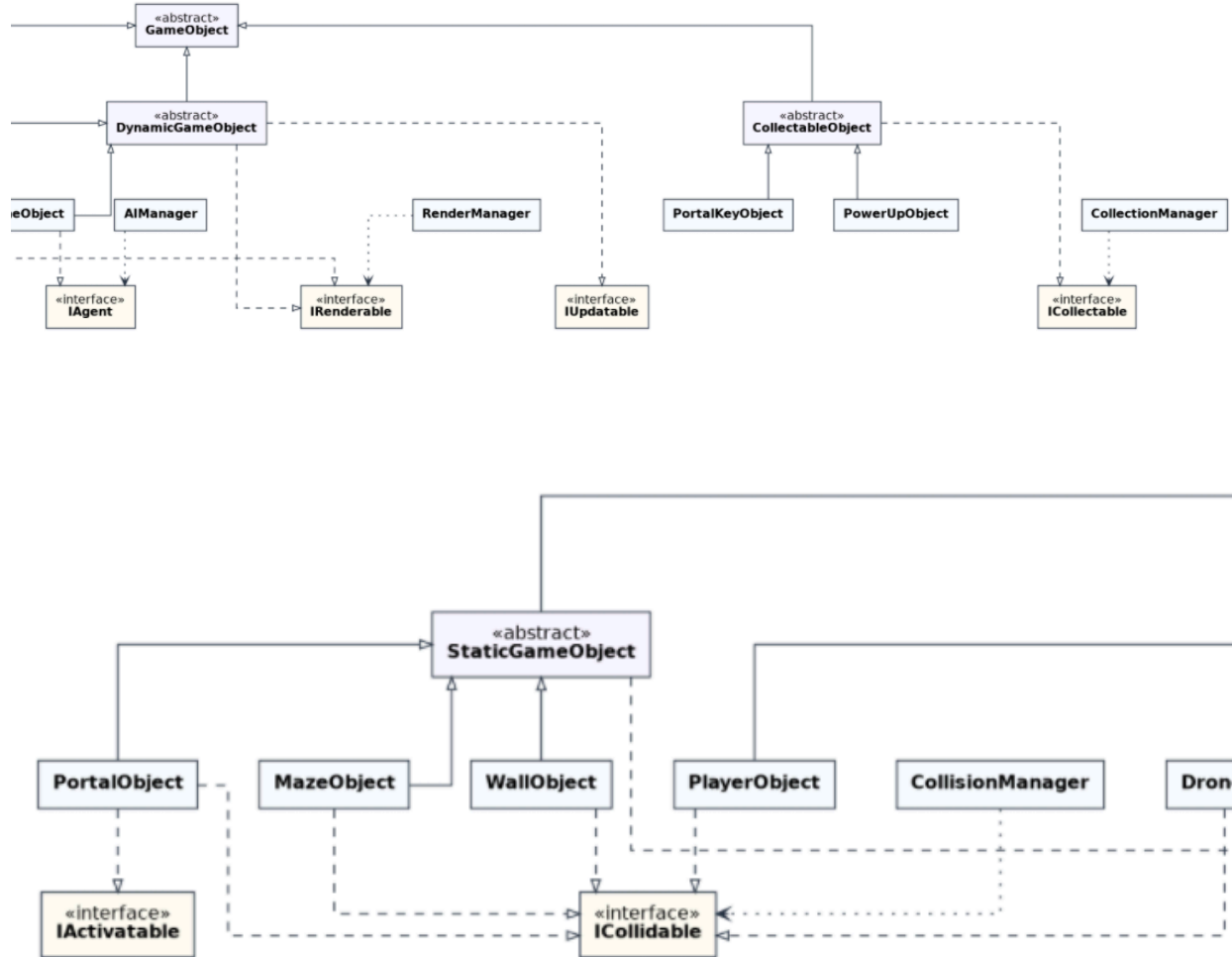
1.7 Improved Inheritance-Based Design

The inheritance example from lab 1 uses a simple object-oriented structure. The main base class is GameObject, which stores common fields such as name, position, velocity, geometry, collision state and collision radius. Classes such as Player, Drone, Maze, Portal and PortalKey inherit from this base class. This design is easy to understand, but it has several weaknesses. All objects inherit the same basic data, even when they do not need it. For example, the maze has velocity even though it does not move. The player has geometry even though the player is represented by the camera in first-person view. The portal and key both inherit the same structure even though their behaviours are different.

The original inheritance design also exposes many fields publicly. This makes the code simple, but it weakens encapsulation because any other class can directly change object state. For a larger game, this could make debugging difficult because changes to position, velocity, collision and geometry could happen from anywhere. The collision design is also basic. CollisionManager works directly with GameObject and checks sphere collision using isCollidable and collisionRadius. This works for simple objects, but it does not scale well for different collision shapes. The maze collision is also left as a placeholder. A better inheritance-based design would still use inheritance, but it would separate object responsibilities more clearly.

Figure 3: Improved Inheritance-Based UML

Improved Inheritance-Based Engine Design



In this improved inheritance design, GameObject would only contain data that every object needs, such as name and transform. Rendering would be moved into IRenderable, collision into ICollidable, collectable behaviour into ICollectable, and AI behaviour into IAgent.

This would improve the original inheritance example because objects would no longer be forced to inherit unrelated behaviour. A maze would be a static renderable and collidable object. A drone would be a dynamic renderable, collidable and agent object. A portal key would be a collectable object. A portal would be an activatable object. This is still not as flexible as ECS, but it is a better inheritance design because it reduces the amount of unnecessary shared data in the base class and uses interfaces to represent optional behaviours.

1.8 Comparison of Inheritance and ECS

The improved inheritance design is simpler than ECS. It is easier to understand because object types are represented directly as classes. For a small game, this can be useful because a PlayerObject, DroneObject and PortalObject are easy to identify. However, inheritance becomes

weaker as the number of object behaviours increases. If a new object needs a mixture of features, such as a moving wall that can be collected, damaged and activated, it may not fit neatly into the inheritance tree. This can lead to deeper inheritance chains or repeated code.

ECS is more abstract because an entity is not locked into one category. An object is built by adding components. A moving wall can simply be created with transform, render, collision and velocity components. A power-up can be created with transform, render, collision and gameplay tag components. Existing systems can then process those components without needing a new subclass.

In terms of modularity, ECS is stronger. Movement, rendering, collision and audio are separated into systems and services. Inheritance can be modular if interfaces are used, but it is easier for behaviour to become mixed into base classes. In terms of simplicity, inheritance is stronger at the beginning. It requires less setup and is easier to explain. ECS requires more classes, managers and systems, which makes it more complex to build. In terms of recyclability, ECS is stronger. Components such as transform, velocity, render and collision can be reused in other games. Systems such as movement and rendering can also be reused. Inheritance designs are more likely to become tied to the specific game because classes such as DroneObject and PortalObject contain game-specific behaviour. Overall, the ECS design is better for a reusable game engine, while inheritance is better for a small teaching example or a simple prototype.

Chapter 2: Evaluation Report

2.1 Highscore Client and Server

The game uses a TCP client/server highscore system. The client is implemented in the HighscoreClient class and connects to a local server using 127.0.0.1 on port 8080. The protocol is simple and text-based. The client sends commands to the server and waits for a response.

Command	Purpose
GET_SCORES	Requests the leaderboard.
`SUBMIT name    score`	

When scores are requested, the server returns a JSON list of highscore entries. Each entry contains a name, score and date. The client deserialises this into a list that the game can display. When submitting a score, the client sends the name and score using the SUBMIT

command. If the server responds with OK, the client then requests the updated leaderboard. This keeps the highscore display up to date after a successful submission. The client uses asynchronous networking so the game does not freeze while waiting for a server response. It also uses timeouts and exception handling. If the server is unavailable, the game does not crash. Instead, the client returns an empty score list or a failed result.

This design is suitable for the coursework because it demonstrates client/server communication clearly without making the networking unnecessarily complicated. The main limitation is that the system is local and does not include authentication, score validation or anti-cheat protection. A full commercial version would need a database-backed server, secure validation and protection against fake submissions.

2.2 AI Algorithm

The main AI algorithm is grid-based pathfinding for the enemy drones. The maze is represented as a two-dimensional walkable grid. Open cells are marked as walkable, while wall cells are blocked. Each drone converts its world position into a maze grid cell. The player position is also converted into a grid cell. The drone then uses breadth-first search to find a path from its current cell to the player's cell.

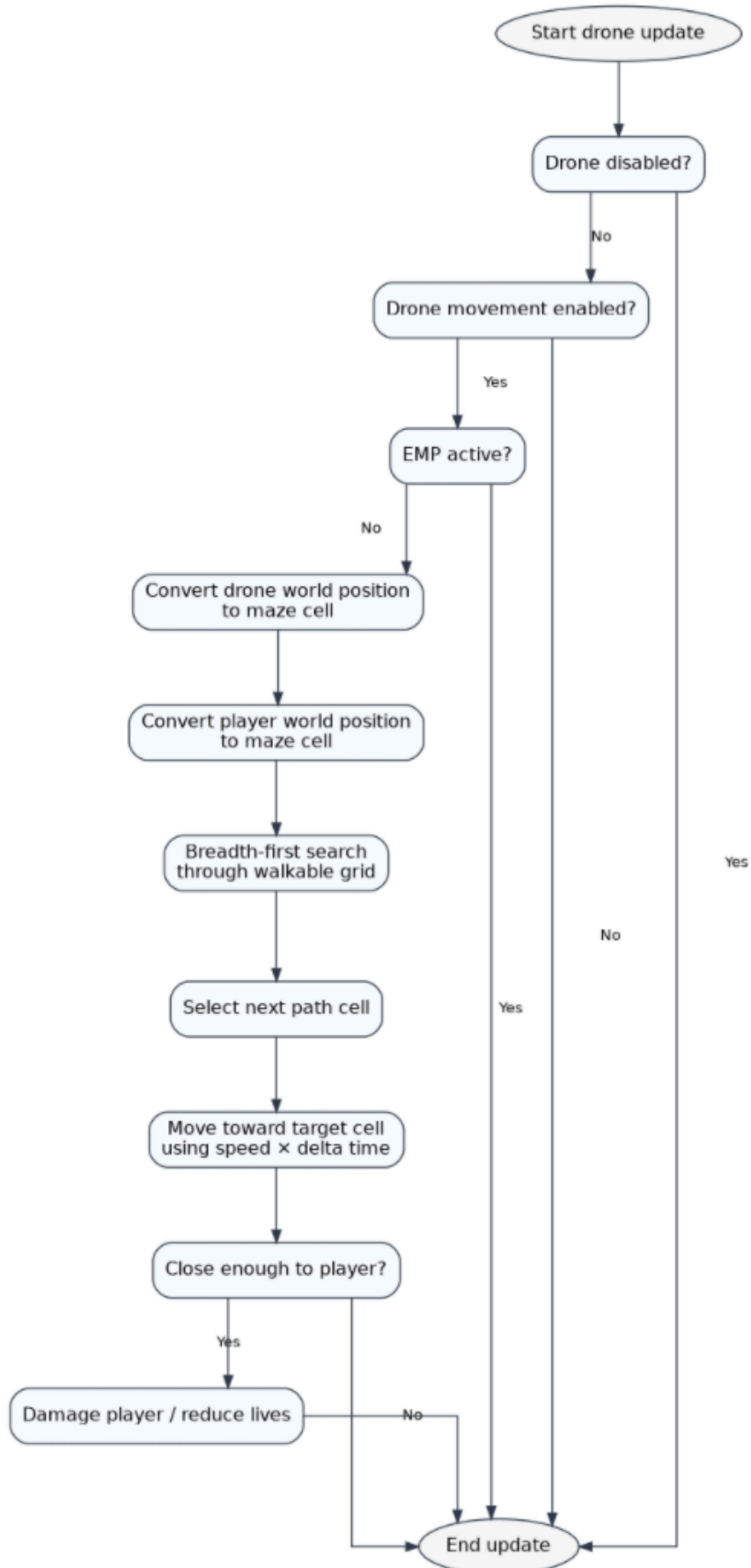
The pathfinding checks four neighbouring cells:

- Right
- Left
- Forward
- Backward

Diagonal movement is not used. This fits the maze structure and prevents drones from cutting through walls. The drone does not need to follow the full path immediately. Instead, it finds the next useful cell and moves toward it using speed and delta time. Once the drone reaches that cell, it calculates the next step toward the player.

Figure 6: Drone AI Flowchart

Drone AI Pathfinding Flowchart



2.3 AI Performance Critique

Breadth-first search is suitable for this game because the maze is small. A 20 by 20 maze only contains 400 cells. Even if the algorithm checks every cell, the cost is still low.

The time complexity is:

$$O(V + E)$$

V is the number of grid cells and E is the number of connections between cells. Each cell has a maximum of four neighbours, so the number of connections is limited.

The memory cost is also acceptable because the algorithm only needs a queue, a visited grid and previous-cell tracking. However, if the maze became much larger or the game had many drones, this could become less efficient. In that case, A* pathfinding would be a better option because it uses a heuristic to search toward the target more directly.

The AI is effective for the submitted game, but it is still simple. The drones mainly chase the player. They do not patrol, search, retreat or coordinate with each other. A future version could improve this by using a finite state machine with states such as Patrol, Chase, Stunned and Disabled.

2.4 Programming Problems and Solutions

Monolithic Game Structure

One of the biggest problems was that the original game scene became too large. Too many responsibilities were handled in one place, including input, rendering, collision, AI, HUD logic, audio and game state. This made the code harder to debug and harder to extend. The attempted solution was to refactor the game into an ECS-style engine. The refactor introduced entities, components, systems, managers and services. For example, rendering was moved into RenderSystem, drone logic into DroneService, power-up logic into PowerUpService, collision into EngineWorldCollisionService, and audio into AudioService.

This was a good design direction, but the refactor was not fully completed for final submission. The main reason was poor time management. I started restructuring the project too late, which meant there was not enough time to fully replace the original implementation and test everything properly. For a future project, I would plan the architecture earlier and complete refactors in smaller stages.

Player Wall Collision

The player could become stuck when moving diagonally into walls. This made movement feel clunky because the player would lose all movement even if only one axis was blocked. The solution was to resolve collision separately on the X and Z axes. If movement on one axis is

blocked, movement on the other axis can still continue. This creates a wall-sliding effect and makes movement feel smoother.

Drone Navigation

The drones originally struggled when walls were between them and the player. Direct chase movement caused them to push into walls instead of finding a path around them. The solution was to use grid-based pathfinding. By using the maze's walkable grid and breadth-first search, drones can move around obstacles and follow corridors. This made the enemies feel more consistent and improved the challenge of the game.

Rendering Order

During development, rendering logic was briefly placed in the wrong stage of the loop. If objects are rendered before the screen is cleared, the frame can be wiped afterwards. The solution was to keep rendering inside the render method after `GL.Clear`. This follows the correct structure of updating the game state first, clearing the screen, drawing the current frame, and then presenting it.

Resource Management

Repeatedly loading geometry or creating shaders can waste memory and cause performance issues. The solution was to use cached resource loading through the resource manager and shader caching in the render system. This reduces repeated OpenGL resource creation.

Audio Organisation

Managing OpenAL source IDs directly inside the scene made the code harder to maintain. The solution was to create `AudioService`, which registers and plays sounds by name. This keeps the scene cleaner and makes audio easier to manage.

2.5 Final Evaluation

The final ECS design is a strong improvement over the original inheritance-based and scene-heavy structure. It separates data, behaviour and services more clearly. This makes the project easier to maintain and gives the engine a better foundation for future projects.

The strongest parts of the design are:

- ECS-style entities and components
- Separate systems for movement and rendering
- Separate services for drones, power-ups, collision, audio and highscores
- Grid-based drone pathfinding
- AABB wall collision
- Cached geometry, audio and shader resources

The main weakness is that the refactor was not completed in time for the final submission. Some responsibilities still remain concentrated in RefactorGameScene, and the ECS implementation is still basic. Component lookup uses simple lists, which is acceptable for this project but would not be ideal for a larger engine.

The AI is effective for a small maze, but it could be improved with A* pathfinding and finite state behaviour. The physics is also limited to velocity movement and AABB collision rather than full rigid body simulation. Overall, the project demonstrates a clear move toward a more modular engine design. The ECS structure is more flexible and reusable than the original inheritance approach, even though the refactor was not fully completed.

2.6 Conclusion

This project demonstrates a first-person maze survival game supported by an ECS-style engine design. The engine uses entities, components, systems, managers and services to separate responsibilities. The inheritance example from lab 1 was useful for comparison because it showed the limitations of a simple base-class design. The improved inheritance design is easier to understand than ECS, but ECS is better for modularity, abstractness and recyclability. The game includes a TCP highscore client, drone pathfinding, AABB collision, power-ups, audio handling, HUD rendering and configurable settings. The main limitation is that the ECS refactor was not completed in time due to poor time management. However, the refactor still shows a clear understanding of how the game engine could be improved and extended in future.